Materi Hari 1 Dart dan Flutter UI

Flutter Android untuk tim developer PT. Dok Pantai Lamongan

Trainer dan Pembuat Buku: Damar Galih Aji Pradana, S.Tr.Kom.

Target hari ini

Bangun daftar inspeksi, detail, dan form yang berjalan di iPhone 16 Simulator dengan data lokal. Setelah sesi ini peserta dapat membaca Dart, menyusun widget berdasarkan constraint, mengelola state lokal, serta menjelaskan bagaimana form mengembalikan data ke daftar. Gunakan PRD Hari 1 sebagai kriteria hasil, bukan sekadar meniru screenshot trainer.

> **Proyek resmi:** `flutter_projects/hari_1_dpl_inspection`, hasil lengkap `flutter create` dengan Android+iOS. Gunakan Flutter 3.44.8 stable dan iPhone 16 Simulator iOS 18.0. Android tetap tersedia, tetapi kelas cukup menjalankan satu simulator agar penggunaan RAM lebih ringan.



Hasil Hari 1 — daftar inspeksi pada iPhone 16

Peta belajar dan checkpoint

Waktu	Penjelasan dan praktik	Bukti selesai
08.30-10.00	Workflow Flutter dan Dart yang dibutuhkan proyek	App berjalan; model dan 3 record
10.15-12.00	Widget, layout, list, theme dan responsive UI	Daftar dan detail tanpa overflow
13.00-14.30	Form, validation, Navigator dan setState	Record baru muncul; input invalid ditolak
14.45-16.00	Komponen reusable, lifecycle dan debugging	Empty/error/retry dan controller dispose
16.00-16.30	Acceptance review dan Git	Bukti uji serta tag day-1

Sebelum membuka editor

Flutter adalah toolkit UI dengan Dart sebagai bahasa aplikasi. Widget mendeskripsikan konfigurasi UI. Ketika state berubah, framework membangun ulang bagian tree yang perlu; build bukan tempat menjalankan request, membuat file, atau menjalankan operasi dengan side effect.

Bandingkan dengan pengalaman Kotlin: data class dapat dipetakan ke model Dart, suspend workflow ke Future/async, dan state UI deklaratif ke widget yang dibangun dari data. Pemetaan ini membantu belajar, tetapi lifecycle dan layout Flutter tetap perlu dipahami sendiri.

```
flutter --version
dart --version
flutter doctor -v
flutter devices
cd flutter_projects/hari_1_dpl_inspection
fvm flutter pub get
fvm flutter run -d "iPhone 16"
```

Catat SDK/device yang dipakai di README. Hot reload umumnya mempertahankan state; hot restart membuat ulang aplikasi Dart. Setelah perubahan native atau pemasangan plugin, hentikan aplikasi dan run ulang. Jangan memakai flutter clean sebagai solusi pertama untuk setiap error.

Persiapan: IDE dengan Flutter/Dart extension, Flutter 3.44.8, Xcode, iPhone 16 Simulator, dan Git sudah siap. Trainer memulai dari proyek lengkap agar waktu kelas digunakan untuk memahami dan mengubah Flutter, bukan menunggu scaffolding.

Dart yang langsung dipakai di proyek

Kebutuhan	Contoh Dart	Penjelasan
Nilai tidak berubah	final assetCode = ‘PMP-001’;	final diisi satu kali; const adalah nilai compile-time
Nullable	String? note;	Tanda ? memperbolehkan null
Default saat null	note ?? ’’	Gunakan fallback; jangan menyebar operator !
Named parameter	Inspection(assetCode: ‘PMP-001’)	Membuat pemanggilan jelas
Collection	`List<Inspection>` dan `Map<String, dynamic>`	JSON perlu parsing ke model yang bertipe
Async	`Future<List<Inspection>>`	Hasil datang kemudian; await berada di fungsi async

Model pertama

```dart
class Inspection {
  final String id;
  final String assetCode;
  final String location;
```

```
  final String condition;
  final String note;
  final DateTime createdAt;

  const Inspection({required this.id, required this.assetCode,
    required this.location, required this.condition,
    required this.note, required this.createdAt});
}
```

Validasi sebagai fungsi murni

```
String? validateNote(String condition, String raw) {
  final note = raw.trim();
  if (note.length > 500) return 'Maksimal 500 karakter';
  if (condition == 'needs_action' && note.length < 10) {
    return 'Temuan membutuhkan catatan minimal 10 karakter';
  }
  return null;
}
```

Fungsi mengembalikan null bila valid sehingga dapat dipakai sebagai validator form. Fungsi murni mudah dites dan tidak bergantung pada widget.

Latihan 1

1. Buat 3 Inspection dengan kondisi berbeda; simpan dalam List.
2. Tampilkan jumlah record needs_action menggunakan where().length.
3. Buat fungsi async yang menunggu 600 ms lalu mengembalikan daftar.
4. Jelaskan mengapa Future bukan List dan mengapa tidak aman memakai ! untuk semua nullable field.

Hasil: model, seed data dan validator dapat dibaca tanpa context UI. Jangan mengerjakan inheritance atau generics kompleks yang tidak diperlukan latihan.

Widget tree dan aturan layout

Scaffold menyediakan struktur layar; AppBar judul; body berisi konten. Row/Column menata satu arah, Padding memberi ruang, Expanded membagi sisa ruang, dan ListView.builder membuat item sesuai kebutuhan scroll. Pilih widget berdasarkan tugasnya agar tree tetap jelas.

Constraint flow: parent memberi batas ukuran, child memilih ukuran di dalam batas tersebut, lalu parent menempatkan child. Error layout sering muncul karena child diminta membesar dalam arah yang tidak mempunyai batas, bukan karena padding kurang.

```
Scaffold(
  appBar: AppBar(title: const Text('Daftar inspeksi')),
  body: Column(children: [
    const Padding(
      padding: EdgeInsets.all(16),
      child: Text('Pemeriksaan peralatan latihan'),
    ),
```

```
    Expanded(child: ListView.builder(
      itemCount: items.length,
      itemBuilder: (context, index) {
        final item = items[index];
        return ListTile(
          key: ValueKey(item.id),
          title: Text(item.assetCode),
          subtitle: Text(item.location),
          trailing: Text(item.condition == 'good'
              ? 'Baik' : 'Perlu tindak lanjut'),
          onTap: () => openDetail(item),
        );
      },
    )),
  ]),
)
```

Potongan ini diletakkan dalam build layar daftar; items dan openDetail disediakan oleh state/layar. Contoh starter lengkap ada di folder praktik/hari_1.

Latihan 2

1. Bangun daftar memakai ListView.builder; gunakan item.id sebagai key.
2. Buat detail yang menampilkan kode, lokasi, kondisi, catatan dan waktu.
3. Gunakan SingleChildScrollView untuk detail panjang dan form, bukan menggabungkan dua scroll vertikal tanpa alasan.
4. Uji landscape dan lebar 360 logical pixels; wrap teks panjang.

Diskusi: Expanded di dalam Column berbounded membuat list mengambil sisa tinggi. Expanded di dalam SingleChildScrollView pada arah vertikal dapat gagal karena tinggi tidak dibatasi. Perbaiki parent constraint, bukan memberi fixed height acak pada semua widget.

State lokal dan lifecycle

StatelessWidget sesuai untuk komponen yang tampil dari input. StatefulWidget dipakai ketika layar mempunyai data berubah, controller, selection atau lifecycle. State tidak identik dengan seluruh data bisnis; teks form, tombol sedang diproses, dan selection juga state.

```
Future<void> load() async {
  setState(() { isLoading = true; errorMessage = null; });
  try {
    final result = await loadDemoItems();
    if (!mounted) return;
    setState(() => items = result);
  } catch (_) {
    if (!mounted) return;
    setState(() => errorMessage = 'Data gagal dimuat');
  } finally {
    if (mounted) setState(() => isLoading = false);
```

```
  }
}
```

initState menjalankan setup sekali, build dapat dipanggil berulang, dan dispose melepas resource milik state. Callback setState sendiri harus sinkron; operasi async dilakukan di luar callback seperti contoh.

Empat state daftar

- Loading: progress indicator dengan label proses; jangan menampilkan empty sekaligus.
- Success: daftar record. Jika result kosong, tampilkan empty state yang memberi tombol tambah.
- Error: pesan yang dapat dipahami dan tombol retry. Pertahankan data lama hanya jika kebijakan UI jelas.
- Empty: request/loading sudah selesai dengan hasil benar-benar kosong, bukan akibat request gagal.

Latihan 3

Tambahkan loading simulasi 600 ms, tombol demo error sekali, retry, serta tombol reset ke daftar kosong. Pastikan hasil form baru tetap terlihat dan tidak langsung ditimpa loading dummy berikutnya.

Lifecycle controller form

```
final assetController = TextEditingController();

@override
void dispose() {
  assetController.dispose();
  super.dispose();
}
```

Jangan membuat TextEditingController setiap build: teks bisa ter-reset dan resource tidak dilepas. Keys menjaga identitas widget saat item berpindah; key bukan cara otomatis memperbaiki desain state yang salah.

Form dan navigation yang terhubung

Navigator.push membuka form/detail. Form mengembalikan Inspection melalui Navigator.pop. Layar daftar menerima hasil dan menambahkannya ke state. Alur ini cukup untuk proyek kecil tanpa routing package tambahan.

```
final created = await Navigator.of(context).push<Inspection>(
  MaterialPageRoute(builder: (_) => const InspectionForm()),
);
if (!mounted || created == null) return;
setState(() => items = [created, ...items]);
```

Membuat form

Gunakan GlobalKey FormState, Form, TextFormField, DropdownButtonFormField, dan tombol simpan. Batasi panjang input, trim sebelum validasi/simpan, dan simpan enum string yang konsisten untuk JSON nanti.

```
final formKey = GlobalKey<FormState>();

TextFormField(
  controller: assetController,
  maxLength: 30,
  decoration: const InputDecoration(labelText: 'Kode aset'),
  validator: (value) => (value ?? '').trim().isEmpty
      ? 'Kode aset wajib diisi' : null,
)
```

Langkah praktik 4

1. Buat field kode aset maksimal 30, lokasi maksimal 80, kondisi dan catatan maksimal 500 karakter.
2. Saat tombol simpan ditekan, panggil formKey.currentState!.validate(). Penggunaan ! di sini terbatas pada key Form yang sedang terpasang.
3. Jika kondisi needs_action, panggil validateNote dan tampilkan error bila catatan kurang dari 10 karakter.
4. Jika valid, buat ID dengan timestamp mikrodetik untuk latihan lokal dan waktu DateTime.now().toUtc().
5. Navigator.pop(context, record). Daftar menerima record; tombol batal hanya pop tanpa hasil.
6. Uji dua kali simpan, input spasi, batal setelah mengisi dan kembali dengan tombol Android.

Expected result: catatan valid muncul sekali, nilai detail sama dengan form, dan pembatalan tidak membuat data. Hari 2 akan mengganti ID lokal dengan ID server dan mencegah submit ganda saat request.

Theme, responsive UI dan komponen reusable

Pakai ThemeData dan ColorScheme.fromSeed agar text/button mempunyai style konsisten. Ambil typography melalui Theme.of(context).textTheme. Pisahkan widget seperti InspectionTile dan StatusBadge ketika komponen memang dipakai berulang atau memiliki tanggung jawab jelas.

Responsive layout dimulai dari constraint. Gunakan LayoutBuilder untuk memilih satu atau dua kolom berdasarkan ruang yang tersedia; gunakan MediaQuery untuk informasi perangkat yang relevan. Hindari mengukur semua elemen dengan persentase layar jika widget dapat menyesuaikan diri secara alami.

```
MaterialApp(
  theme: ThemeData(
```

```
    colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
    useMaterial3: true,
  ),
  home: const InspectionListScreen(),
)
```

Praktik perbaikan kualitas

- Tambahkan SafeArea bila konten berpotensi masuk area sistem; jangan menggandakan padding tanpa memeriksa tampilan.
- Form dapat di-scroll saat keyboard muncul. Teks error tidak tertutup tombol simpan.
- Kondisi memakai teks/icon selain warna sehingga tetap dapat dibedakan.
- Loading dan tombol disabled mempunyai label yang jelas.
- Long text dibungkus; gunakan Flexible/Expanded pada Row yang memuat teks panjang.

Git checkpoint

```
git init
git add .
git commit -m "Complete day 1 Flutter UI and form"
git tag day-1
```

Sebelum commit, periksa .gitignore template, daftar file yang staged, dan tidak adanya credential. Bagi praktik berpasangan: satu peserta mengetik, satu memeriksa state/acceptance; tukar peran setelah satu sesi.

Review kode singkat

Apakah build mempunyai side effect? Apakah controller di-dispose? Apakah model dan validator dapat dipakai tanpa widget? Apakah state form dan state daftar mempunyai pemilik yang jelas? Peserta harus bisa menjelaskan satu keputusan, bukan hanya menunjukkan app berjalan.

Debugging dan evaluasi hari 1

Gejala	Penyebab yang diperiksa	Tindakan
RenderFlex overflow	Teks/Row terlalu lebar; constraint	Flexible, wrap, inspeksi parent
Vertical viewport unbounded	ListView dalam Column tanpa batas	Expanded pada parent yang tepat
Form ter-reset	Controller dibuat di build	Pindahkan ke State; dispose
setState after dispose	Async selesai setelah halaman ditutup	Periksa mounted dan lifecycle
Duplikasi item	Save dipanggil dua kali/daftar diubah dua tempat	Tentukan satu pemilik mutation
Hot reload tidak mengubah initial state	initState tidak dijalankan ulang	Hot restart bila diperlukan

Gunakan Widget Inspector untuk membaca tree dan constraints. Breakpoint ditempatkan pada validator, callback save dan penerimaan hasil Navigator. Baca error pertama yang menjelaskan sebab, bukan hanya baris terakhir log.

Evaluasi pemahaman

1. Mengapa build tidak boleh memulai API request setiap kali dipanggil?
2. Kapan local state cukup dan kapan data perlu dikelola di luar layar?
3. Mengapa Future berbeda dari Inspection?
4. Apa perbedaan empty dan error?
5. Mengapa controller perlu dispose dan mounted diperiksa setelah await?

Jawaban ringkas untuk trainer: build dapat berulang; shared/async business state membutuhkan pemilik terpisah; Future mewakili hasil mendatang; empty adalah hasil valid tanpa item; resource/lifecycle UI harus sesuai masa hidup widget.

Tugas akhir hari

Jalankan AC1-AC8 pada PRD Hari 1. Serahkan source day-1 dan bukti daftar-detail-form invalid/form valid. Tambahan search lokal boleh dikerjakan bila semua kriteria inti lulus. Jangan pindah ke package state management sebelum peserta memahami aliran data local state.

Referensi resmi dan batas contoh

- Dart language: https://dart.dev/language
- Null safety: https://dart.dev/null-safety
- Async: https://dart.dev/language/async
- Flutter UI: https://docs.flutter.dev/ui
- Navigation: https://docs.flutter.dev/cookbook/navigation

Materi dan kode latihan disusun untuk kasus ini. Contoh Dart/Flutter belum dikompilasi di lingkungan penyusunan karena Flutter SDK tidak tersedia; trainer menjalankan analyze/test pada toolchain kelas sebelum digunakan. Referensi diperiksa 03 Oktober 2026.